

AI Assisted Coding

Assignment –16.3

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Lab16.sql

```
Run on active connection | Select block
1  -- Design a normalized relational database schema for a Hospital Management System using SQL.
2  --
3  -- Requirements:
4  -- - Tables: Doctors, Patients, Appointments
5  -- - Include primary keys, foreign keys, constraints (unique IDs, valid dates)
6  -- - Ensure normalization (up to 3NF)
7  --
8  -- Generate SQL queries:
9  -- 1. List all appointments for a specific doctor
10 -- 2. Retrieve patient history by patient ID
11 -- 3. Count total patients treated by each doctor
12 --
13 -- - Use proper JOINS
14 -- - Include sample INSERT data
15 -- - Add comments explaining schema and queries
16 --
17 -- Output complete SQL script.
18 CREATE TABLE Doctors (
19     doctor_id INT PRIMARY KEY,
20     name VARCHAR(255) NOT NULL,
21     specialization VARCHAR(255) NOT NULL
22 );
23 CREATE TABLE Patients (
24     patient_id INT PRIMARY KEY,
25     name VARCHAR(255) NOT NULL,
26     date_of_birth DATE NOT NULL
27 );
28 CREATE TABLE Appointments (
29     appointment_id INT PRIMARY KEY,
30     doctor_id INT,
31     patient_id INT,
32     appointment_date DATE NOT NULL,
33     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
34     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
35 );
36 INSERT INTO Doctors (doctor_id, name, specialization) VALUES
37     (1, 'Dr. John Doe', 'Cardiologist'),
38     (2, 'Dr. Jane Smith', 'Pediatrician'),
39     (3, 'Dr. Michael Brown', 'Dermatologist');
40 INSERT INTO Patients (patient_id, name, date_of_birth) VALUES
41     (1, 'John Doe', '1980-01-01'),
42     (2, 'Jane Smith', '1990-05-15'),
43     (3, 'Michael Brown', '1985-07-20');
44 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
45     (1, 1, 1, '2024-01-01'),
46     (2, 2, 2, '2024-02-01'),
47     (3, 3, 3, '2024-03-01');
48 -- List all appointments for a specific doctor
49 SELECT * FROM Appointments WHERE doctor_id = 1;
50 -- Retrieve patient history by patient ID
51 SELECT * FROM Patients WHERE patient_id = 1;
52 -- Count total patients treated by each doctor
53 SELECT doctor_id, COUNT(*) AS total_patients FROM Appointments GROUP BY doctor_id;
54
```

```

1 """Load Lab16.sql and run it with SQLite (stdlib only)."""
2 import sqlite3
3 import sys
4 import traceback
5 from pathlib import Path
6
7 HERE = Path(__file__).resolve().parent
8 SQL_FILE = HERE / "Lab16.sql"
9 DB_FILE = HERE / "Lab16_hospital.db"
10
11
12 def _is_sql_comment_line(s: str) -> bool:
13     t = s.strip()
14     return not t or t.startswith("--") or t.startswith("#")
15
16
17 def first_sql_keyword(stmt: str) -> str:
18     for line in stmt.splitlines():
19         s = line.strip()
20         if _is_sql_comment_line(s):
21             continue
22         return s.split(None, 1)[0].upper()
23     return ""
24
25
26 def run_sql_file(sql_path: Path, db_path: Path) -> None:
27     def log(msg: str) -> None:
28         print(msg, flush=True)
29
30     sql = sql_path.read_text(encoding="utf-8")
31     log(f"Loading SQL: {sql_path}")
32     if db_path.exists():
33         db_path.unlink()
34     conn = sqlite3.connect(db_path)
35     conn.execute("PRAGMA foreign_keys = ON")
36     cur = conn.cursor()
37
38     parts = [p.strip() for p in sql.split(";") if p.strip()]
39     setup, selects = [], []
40     for chunk in parts:
41         (selects if first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
42
43     log(f"Running {len(setup)} DDL/DML statement(s)...")
44     for stmt in setup:
45         cur.execute(stmt + ";")
46     conn.commit()
47
48     if not selects:
49         log("No SELECT statements found after splitting on ';'. Check Lab16.sc
50
51
52 def run_sql_file(sql_path: Path, db_path: Path) -> None:
53     ...
54     (selects if first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
55     log(f"Running {len(setup)} DDL/DML statement(s)...")
56     for stmt in setup:
57         cur.execute(stmt + ";")
58     conn.commit()
59
60     if not selects:
61         log("No SELECT statements found after splitting on ';'. Check Lab16.sc
62
63     for i, stmt in enumerate(selects, start=1):
64         log(f"\n--- Query {i} ---")
65         rows = cur.execute(stmt + ";").fetchall()
66         if not rows:
67             log("no rows")
68         for row in rows:
69             log(str(row))
70     conn.close()
71     log(f"\nDatabase saved: {db_path}")
72
73
74 def main() -> None:
75     if not SQL_FILE.is_file():
76         print(f"Missing SQL file: {SQL_FILE}", file=sys.stderr, flush=True)
77         raise SystemExit(1)
78     run_sql_file(SQL_FILE, DB_FILE)
79
80
81 if __name__ == "__main__":
82     try:
83         print("Lab16: starting...", flush=True)
84         main()
85         print("Lab16: finished.", flush=True)
86     except Exception:
87         traceback.print_exc()
88         raise SystemExit(1)

```

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

```

app.py Lab16sql x lab16.py + lab16_last_run.txt SQLTools Set ... Lab16sql x
Lab16sql
2 -- Design a normalized SQL database schema for an E-commerce platform.
3
4 -- Requirements:
5 -- - Tables: Users, Products, Orders, OrderDetails
6 -- - Include primary keys, foreign keys, constraints
7 -- - Ensure normalization (up to 3NF)
8
9 -- Generate SQL queries:
10 -- 1. Retrieve all orders by a user
11 -- 2. Find the most purchased product
12 -- 3. Calculate total revenue in a given month
13
14 -- - Use proper JOINs and aggregate functions
15 -- - Include sample INSERT data
16 -- - Suggest normalization improvements
17 -- - Suggest query optimization techniques
18
19 -- Output complete SQL script with comments.
20
21 CREATE TABLE Users (
22     user_id INT PRIMARY KEY,
23     name VARCHAR(255) NOT NULL,
24     email VARCHAR(255) NOT NULL UNIQUE,
25     password VARCHAR(255) NOT NULL
26 );
27
28 CREATE TABLE Products (
29     product_id INT PRIMARY KEY,
30     name VARCHAR(255) NOT NULL,
31     price DECIMAL(10, 2) NOT NULL
32 );
33
34 CREATE TABLE Orders (
35     order_id INT PRIMARY KEY,
36     user_id INT,
37     product_id INT,
38     quantity INT NOT NULL,
39     order_date DATE NOT NULL,
40     FOREIGN KEY (user_id) REFERENCES Users(user_id),
41     FOREIGN KEY (product_id) REFERENCES Products(product_id)
42 );
43
44 CREATE TABLE OrderDetails (
45     order_detail_id INT PRIMARY KEY,
46     order_id INT,
47     product_id INT,
48     quantity INT NOT NULL,
49     price DECIMAL(10, 2) NOT NULL,
50     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
51     FOREIGN KEY (product_id) REFERENCES Products(product_id)
52 );
53
54 INSERT INTO Users (user_id, name, email, password) VALUES
55 (1, 'John Doe', 'john.doe@example.com', 'password123'),
56 (2, 'Jane Smith', 'jane.smith@example.com', 'password456'),
57 (3, 'Michael Brown', 'michael.brown@example.com', 'password789');
58
59 INSERT INTO Products (product_id, name, price) VALUES
60 (1, 'Product A', 100.00),
61 (2, 'Product B', 200.00),
62 (3, 'Product C', 300.00);
63
64 INSERT INTO Orders (order_id, user_id, product_id, quantity, order_date) VALUES
65 (1, 1, 1, 1, '2024-01-01'),
66 (2, 2, 2, 2, '2024-02-01'),
67 (3, 3, 3, 3, '2024-03-01');
68
69 INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, price) VALUES
70 (1, 1, 1, 1, 100.00),
71 (2, 2, 2, 2, 200.00),
72 (3, 3, 3, 3, 300.00);
73
74 -- Retrieve all orders by a user
75 SELECT * FROM Orders WHERE user_id = 1;
76
77 -- Find the most purchased product
78 SELECT product_id, SUM(quantity) AS total_quantity FROM Orders GROUP BY product_id
79 ORDER BY total_quantity DESC LIMIT 1;
80
81 -- Calculate total revenue in a given month (line items: quantity * price from
82 SELECT SUM(od.quantity * od.price) AS total_revenue
83 FROM OrderDetails od
84 INNER JOIN Orders o ON od.order_id = o.order_id
85 WHERE o.order_date BETWEEN '2024-01-01' AND '2024-01-31';

```

```

lab16.py + ...
1 """
2 Run Lab16.sql from this folder using SQLite. No MySQL required.
3
4 Terminal: python Lab16.py
5 Unbuffered: python -u Lab16.py
6
7 If the terminal shows nothing, open lab16_last_run.txt (created in this folder).
8 """
9
10 import sqlite3
11 import sys
12 import traceback
13 from pathlib import Path
14
15 HERE = Path(__file__).resolve().parent
16 LAST_RUN_LOG = HERE / "lab16_last_run.txt"
17
18 def _is_sql_comment_line(s: str) -> bool:
19     t = s.strip()
20     return not t or t.startswith("--") or t.startswith("#")
21
22 def _first_sql_keyword(stmt: str) -> str:
23     for line in stmt.splitlines():
24         s = line.strip()
25         if _is_sql_comment_line(s):
26             continue
27         return s.split(None, 1)[0].upper()
28     return ""
29
30 def _resolve_lab16_sql() -> Path:
31     for name in ("lab16.sql", "lab16.py"):
32         p = HERE / name
33         if p.is_file():
34             return p
35     return None
36
37 def _run_sql_file(sql_path: Path, db_path: Path, log) -> None:
38     sql = sql_path.read_text(encoding="utf-8")
39     log(f"Running SQL file: {sql_path}")
40     if db_path.exists():
41         db_path.unlink()
42     conn = sqlite3.connect(db_path)
43     conn.execute("PRAGMA foreign_keys = ON")
44     cur = conn.cursor()
45
46     parts = [p.strip() for p in sql.split(";") if p.strip()]
47     setup, selects = [], []
48     for chunk in parts:
49         (selects if _first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
50
51     log(f"Running (len(setup)) DDL/OQL statement(s)...")
52     for stmt in setup:
53         cur.execute(stmt + ";")
54     conn.commit()
55
56 def _emit(msg: str, log_lines: list[str]) -> None:
57     log(msg)
58     log_lines.append(msg)
59     try:
60         print(msg, flush=True)
61     except Exception:
62         pass
63     sys.stdout.write(msg + "\n")
64     sys.stdout.flush()
65
66 def _run_sql_file(sql_path: Path, db_path: Path, log) -> None:
67     sql = sql_path.read_text(encoding="utf-8")
68     log(f"Running SQL file: {sql_path}")
69     if db_path.exists():
70         db_path.unlink()
71     conn = sqlite3.connect(db_path)
72     conn.execute("PRAGMA foreign_keys = ON")
73     cur = conn.cursor()
74
75     parts = [p.strip() for p in sql.split(";") if p.strip()]
76     setup, selects = [], []
77     for chunk in parts:
78         (selects if _first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
79
80     log(f"Running (len(setup)) DDL/OQL statement(s)...")
81     for stmt in setup:
82         cur.execute(stmt + ";")
83     conn.commit()
84
85 def _main() -> None:
86     sql_path = _resolve_lab16_sql()
87     if not sql_path.is_file():
88         msg = f"SQL file not found: {sql_path}"
89         print(msg, file=sys.stderr, flush=True)
90         LAST_RUN_LOG.write_text(msg + "\n", encoding="utf-8")
91         raise SystemExit(1)
92
93     db_path = sql_path.parent / f"{sql_path.stem}.db"
94     lines: list[str] = []
95
96     def _log(msg: str) -> None:
97         _emit(msg, lines)
98
99     log("Lab16.py started.")
100     try:
101         _run_sql_file(sql_path, db_path, _log)
102         log("Lab16.py finished OK.")
103         log(f"(Copy of this output: {LAST_RUN_LOG})")
104     finally:
105         LAST_RUN_LOG.write_text("\n".join(lines) + "\n", encoding="utf-8")
106
107 if __name__ == "__main__":
108     _main()
109
110 except SystemExit:
111     tb = traceback.format_exc()
112     try:
113         sys.stderr.write(tb)
114     except Exception:
115         pass
116     LAST_RUN_LOG.write_text(tb, encoding="utf-8")
117     raise SystemExit(1)

```

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

```
Lab16.sql
1  -- Design a normalized SQL database schema for a Library Management
2
3  -- Requirements:
4  -- - Tables: Books, Members, Loans
5  -- - Include primary keys, foreign keys, constraints
6  -- - Ensure normalization (up to 3NF)
7
8  -- Generate SQL queries:
9  -- 1. Retrieve all currently issued books
10 -- 2. Find overdue books (loan date > 30 days)
11 -- 3. Count number of books loaned by each member
12
13 -- Use JOINS and conditions
14 -- Include sample INSERT data
15 -- Suggest indexing strategy for performance
16
17 -- Output complete SQL script with comments.
18 -- Create the Books table
19 CREATE TABLE Books (
20     BookID INT PRIMARY KEY AUTO_INCREMENT,
21     Title VARCHAR(255) NOT NULL,
22     Author VARCHAR(255) NOT NULL,
23     PublishedYear INT,
24     Genre VARCHAR(100)
25 );
26
27 -- Create the Members table
28 CREATE TABLE Members (
29     MemberID INT PRIMARY KEY AUTO_INCREMENT,
30     FirstName VARCHAR(100) NOT NULL,
31     LastName VARCHAR(100) NOT NULL,
32     Email VARCHAR(255) UNIQUE NOT NULL,
33     JoinDate DATE NOT NULL
34 );
35
36 -- Create the Loans table
37 CREATE TABLE Loans (
38     LoanID INT PRIMARY KEY AUTO_INCREMENT,
39     BookID INT NOT NULL,
40     MemberID INT NOT NULL,
41     LoanDate DATE NOT NULL,
42     ReturnDate DATE,
43     FOREIGN KEY (BookID) REFERENCES Books(BookID),
44     FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
45 );
46
47 -- Sample INSERT data for Books
48 INSERT INTO Books (Title, Author, PublishedYear, Genre) VALUES
49 ('The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
50 ('To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
51 ('1984', 'George Orwell', 1949, 'Dystopian');
52
53 -- Sample INSERT data for Members
54 INSERT INTO Members (FirstName, LastName, Email, JoinDate) VALUES
55 ('John', 'Doe', 'john.doe@example.com', '2023-01-01'),
56 ('Jane', 'Smith', 'jane.smith@example.com', '2023-01-01');
57
58 -- Sample INSERT data for Loans
59 INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES
60 (1, 1, '2023-01-10', NULL), -- John Doe has loaned 'The Great Gatsby'
61 (2, 2, '2023-01-15', '2023-02-15'); -- Jane Smith has loaned 'To Kill a Mockingbird'
62
63 -- 1. Retrieve all currently issued books
64 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
65 FROM Loans l
66 JOIN Books b ON l.BookID = b.BookID
67 JOIN Members m ON l.MemberID = m.MemberID
68 WHERE l.ReturnDate IS NULL;
69
70 -- 2. Find overdue books (loan date > 30 days)
71 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
72 FROM Loans l
73 JOIN Books b ON l.BookID = b.BookID
74 JOIN Members m ON l.MemberID = m.MemberID
75 WHERE l.ReturnDate IS NULL AND DATEDIFF(CURDATE(), l.LoanDate) > 30;
76
77 -- 3. Count number of books loaned by each member
78 SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
79 FROM Members m
80 LEFT JOIN Loans l ON m.MemberID = l.MemberID
81 GROUP BY m.MemberID;
82
83 -- Indexing strategy for performance
84 -- Create indexes on foreign keys for faster JOIN operations
85 CREATE INDEX idx_book_id ON Loans(BookID);
86 CREATE INDEX idx_member_id ON Loans(MemberID);
87
88 -- Create an index on LoanDate for faster retrieval of overdue book
89 CREATE INDEX idx_loan_date ON Loans(LoanDate);
```



```
app.py lab16.sql lab16.py q1.html lab203.py q2.py lab16.py
lab16.py > run_sql_file
1 """
2 Run Lab16.sql from this folder using SQLite. No MySQL required.
3 Terminal: python Lab16.py
4 Unbuffered: python -u Lab16.py
5 If the terminal shows nothing, open lab16_last_run.txt (created in this folder).
6 """
7 import sqlite3
8 import sys
9 import traceback
10 from pathlib import Path
11 HERE = Path(__file__).resolve().parent
12 LAST_RUN_LOG = HERE / "lab16_last_run.txt"
13 def _is_sql_comment_line(s: str) -> bool:
14     t = s.strip()
15     return not t or t.startswith("--") or t.startswith("#")
16 def _first_sql_keyword(stmt: str) -> str:
17     for line in stmt.splitlines():
18         s = line.strip()
19         if _is_sql_comment_line(s):
20             continue
21         return s.split(None, 1)[0].upper()
22     return ""
23 def _resolve_lab16_sql() -> Path:
24     # Prefer the library schema file when it exists, but still support the
25     # older lab file names so previous usage keeps working.
26     for name in (
27         "LibraryManagementSystem.sql",
28         "LibraryManagementSystem.sql",
29         "Lab16.sql",
30         "lab16.sql",
31     ):
32         p = HERE / name
33         if p.is_file():
34             return p
35     return HERE / "LibraryManagementSystem.sql"
36 def _emit(msg: str, log_lines: list[str]) -> None:
37     log_lines.append(msg)
38     try:
39         print(msg, flush=True)
40     except Exception:
41         try:
42             sys.stdout.write(msg + "\n")
43             sys.stdout.flush()
44         except Exception:
45             pass
46 def run_sql_file(sql_path: Path, db_path: Path, log) -> None:
47     sql = sql_path.read_text(encoding="utf-8")
48     log(f"Running SQL file: {sql_path}")
49     if db_path.exists():
50         db_path.unlink()
51     conn = sqlite3.connect(db_path)
52     conn.execute("PRAGMA foreign_keys = ON")
53     cur = conn.cursor()
54     parts = [p.strip() for p in sql.split(";") if p.strip()]
55     setup, selects = [], []
56     for chunk in parts:
57         (selects if _first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
58     log(f"Running {len(setup)} DDL/DML statement(s)...")
59
60 def run_sql_file(sql_path: Path, db_path: Path, log) -> None:
61     if not selects:
62         log("No SELECT statements found after splitting on ';'")
63     for i, stmt in enumerate(selects, start=1):
64         log(f"--- Query {i} ---")
65         for line in stmt.splitlines():
66             s = line.strip()
67             if s and not _is_sql_comment_line(s):
68                 log(f"SQL: {s}")
69                 break
70         rows = cur.execute(stmt + ";").fetchall()
71         if not rows:
72             log("(no rows)")
73         for row in rows:
74             log(str(row))
75     conn.close()
76     log(f"SQLite database: {db_path}")
77
78 def main() -> None:
79     if sys.platform == "win32" and hasattr(sys.stdout, "reconfigure"):
80         try:
81             sys.stdout.reconfigure(encoding="utf-8", errors="replace")
82             sys.stderr.reconfigure(encoding="utf-8", errors="replace")
83         except Exception:
84             pass
85     sql_path = Path(sys.argv[1]).expanduser().resolve() if len(sys.argv) > 1 else _resolve_lab16_sql()
86     if not sql_path.is_file():
87         msg = f"SQL file not found: {sql_path}"
88         print(msg, file=sys.stderr, flush=True)
89         LAST_RUN_LOG.write_text(msg + "\n", encoding="utf-8")
90         raise SystemExit(1)
91     db_path = sql_path.parent / f"{sql_path.stem}.db"
92     lines: list[str] = []
93     def log(msg: str) -> None:
94         _emit(msg, lines)
95     log("Lab16.py started.")
96     try:
97         run_sql_file(sql_path, db_path, log)
98     except SystemExit:
99         log("Lab16.py finished OK.")
100     log(f"Copy of this output: {LAST_RUN_LOG}")
101     finally:
102         LAST_RUN_LOG.write_text("\n".join(lines) + "\n", encoding="utf-8")
103 if __name__ == "__main__":
104     try:
105         main()
106     except SystemExit:
107         raise
108     except Exception:
109         tb = traceback.format_exc()
110         try:
111             sys.stderr.write(tb)
112             sys.stderr.flush()
113         except Exception:
114             pass
115     LAST_RUN_LOG.write_text(tb, encoding="utf-8")
116     raise SystemExit(1)
```